MANUAL TÉCNICO DE ARQUITECTURA, ESTÁNDARES DE DESARROLLO Y FLUJO DE TRABAJO — (CORE DEL PROYECTO)

Proyecto: Sistema Integral de Gestión Judicial del Estado de Morelos (SIJ Morelos)
Repositorio Base (Template): `sij-core-template`
(`mx.edu.utez:sij-core:1.0-SNAPSHOT`)
Versión del Estándar: 1.0.0
Alcance: Equipos de Oficialía de Partes, Gestión de Audiencias, Tocas (Segunda Instancia) y Gestión de Peritos.

ÍNDICE GENERAL

1. Propósito y Alcance del Documento
2. Arquitectura Global del Sistema y Metodología de Diseño
    a. 2.1. Arquitectura Distribuida Modular con Esquema de Datos Unificado
    b. 2.2. Patrón Arquitectónico Interno (N-Tier MVC + DAO sobre Jakarta EE 10)
    c. 2.3. Metodología de Hojas de Estilo en Cascada (Arquitectura CSS de 3 Capas + Skeletons)
3. Especificación de Tecnologías, Librerías y Dependencias (`pom.xml`)
    a. 3.1. Entorno de Ejecución y Empaquetado
    b. 3.2. Matriz de Dependencias Oficiales y Versiones Exactas
4. Catálogo Detallado de Directorios y Archivos del Proyecto (`sij-core-template`)
    a. 4.1. Árbol Completo del Proyecto en el IDE
    b. 4.2. Raíz del Proyecto y Automatización (`.github/`, `.mvn/`, `pom.xml`)
    c. 4.3. Paquete Núcleo Inmutable (`src/main/java/mx/edu/utez/sij/core/`)
    d. 4.4. Paquetes de Trabajo por Módulo (`controller`, `service`, `dao`, `model`)
    e. 4.5. Recursos y Base de Datos (`src/main/resources/`)
    f. 4.6. Capa de Presentación Web (`src/main/webapp/`)
5. Política de Archivos Protegidos y Gobernanza Colaborativa (`.github/CODEOWNERS`)
    a. 5.1. Propósito de la Protección del Core
    b. 5.2. Tabla Oficial de Rutas Protegidas y Procedimiento de Solicitud de Cambios
6. Configuración del Entorno Local, Base de Datos y Pruebas de Integración
    a. 6.1. Configuración de `db.properties` y Semilla Automática (*Bootstrap Admin*)
    b. 6.2. Cómo Levantar y Probar el Proyecto en Apache Tomcat 10.1.x
    c. 6.3. Cómo Probar Endpoints entre Módulos Distintos (Postman, Multi-WAR y `ngrok`)
7. Estrategia de Ramas en Git (Gitflow Adaptado) y División del Trabajo sin Conflictos
    a. 7.1. Propósito de las Ramas Principales (`main` y `develop`)
    b. 7.2. Nomenclatura Estándar de Ramas por Equipo, Integrante y Tarea

    c. 7.3. Cómo Dividir las Tareas en el Equipo para Evitar Choques (Vertical Slicing)
    d. 7.4. Reglas de Pull Requests (PR) y Autorización por el Representante de Equipo
8. Estándar de Commits: Guía Completa de Conventional Commits
    a. 8.1. Estructura Formal y Uso de Corchetes `[scope]`
    b. 8.2. Catálogo de Etiquetas (`feat`, `fix`, `style`, `refactor`, `docs`, `chore`, `test`)
9. Metodología Ágil Scrumban / Kanban para la Gestión de Equipos
    a. 9.1. Estructura del Tablero Kanban de 5 Columnas y Límites WIP
    b. 9.2. Flujo de una Tarea desde el Backlog hasta el Merge en `develop`
10. Guía de Buenas Prácticas

---

1. PROPÓSITO Y ALCANCE DEL DOCUMENTO

Su objetivo es que cualquier integrante de los cuatro equipos de desarrollo (**Oficialía de Partes**, **Gestión de Audiencias**, **Tocas** y **Gestión de Peritos**) comprenda con total precisión:

- Para qué sirve cada carpeta y cada archivo del proyecto.
- Qué componentes pertenecen al núcleo compartido del sistema y cuáles son los espacios de trabajo propios de cada módulo.
- Cómo configurar y probar su entorno local sin exponer credenciales sensibles ni romper el trabajo de otros equipos.
- Cómo estructurar sus ramas de Git, redactar commits bajo el estándar internacional **Conventional Commits**, gestionar sus tareas mediante **Kanban** y realizar entregas limpias mediante **Pull Requests**.

---

2. ARQUITECTURA GLOBAL DEL SISTEMA Y METODOLOGÍA DE DISEÑO

2.1. Arquitectura Distribuida Modular con Esquema de Datos Unificado

Para permitir que los 4 equipos trabajen de manera simultánea e independiente sin pisar su código, el proyecto implementa una **Arquitectura Modular de 4 Repositorios** derivados de la plantilla `sij-core-template`, conectados a un **único esquema relacional compartido en MySQL (`sij_db`)** y comunicados entre sí mediante **Endpoints HTTP/JSON (Servlets RESTful)**:

1. `sij-oficialia`: Recepción de demandas y promociones, asignación automática/algorítmica de juzgado, generación de folio/expediente y radicación inicial.
2. `sij-audiencias`: Programación de salas, agenda de audiencias orales, control de asistencia de partes y registro de videograbaciones/actas.
3. `sij-tocas`: Gestión de recursos de apelación en Segunda Instancia, turno a magistrados ponentes y resoluciones.
4. `sij-peritos`: Padrón de peritos auxiliares de la administración de justicia, asignación aleatoria/técnica a expedientes y entrega de dictámenes periciales.

Todos los módulos leen y escriben sobre la misma base de datos local o de integración (`sij_db`) inicializada automáticamente por `schema.sql`, garantizando integridad referencial en catálogos universales (`materias`, `juzgados`, `usuarios`, `expedientes`).

2.2. Patrón Arquitectónico Interno (N-Tier MVC + DAO sobre Jakarta EE 10)

Dentro de cada repositorio, el código fuente sigue estrictamente una **Arquitectura en Capas (N-Tier MVC + Patrón DAO)** con separación de responsabilidades (Separation of Concerns):

- **Capa de Presentación (`webapp/views/` y `webapp/assets/`):** Vistas HTML5 semánticas, estilos CSS modulares y lógica cliente con JavaScript (`fetch()` asíncrono) que consume los Servlets mediante JSON.
- **Capa de Seguridad y Filtros (`core/filter/`):** Interceptores HTTP (`AuthFilter` y `CorsFilter`) que validan sesiones activas, controlan el acceso por rol (RBAC) e inyectan cabeceras de seguridad antes de que la petición llegue a cualquier controlador.
- **Capa de Controladores (`controller/`):** Clases `HttpServlet` que reciben peticiones HTTP (`GET`, `POST`, `PUT`, `DELETE`), extraen parámetros o cuerpos JSON con `Gson`, invocan la capa de negocio y devuelven respuestas JSON estandarizadas mediante `JsonResponseUtil`.
- **Capa de Negocio (`service/`):** Clases Java puras que contienen las reglas de negocio judiciales (validación de plazos, reglas de radicación, sorteo de peritos o salas) antes de tocar la base de datos.
- **Capa de Acceso a Datos (`dao/`):** Clases encargadas exclusivamente de ejecutar sentencias SQL parametrizadas (`PreparedStatement`) usando el pool/conexión provisto por `ConexionMySQL`.
- **Capa de Modelo (`model/`):** Clases POJO / JavaBeans que representan las entidades de las tablas de la base de datos y los objetos de transferencia (DTOs).

2.3. Metodología de Hojas de Estilo en Cascada (Arquitectura CSS de 3 Capas + Skeletons)

Para garantizar que los 4 módulos luzcan como un único sistema gubernamental institucional sin que ningún equipo altere el diseño global por accidente, se aplica la **Metodología de Cascada en 3 Capas con Variables Nativas CSS (Custom Properties)** y nomenclatura modular tipo BEM:

Capa 1 — Tokens de Diseño (`assets/css/variables.css`) [PROTEGIDO]: Define la paleta oficial del Poder Judicial (tonos institucionales, colores de alerta/éxito, tipografía, radios de borde, sombras y espaciados) dentro de `:root`. Nadie debe escribir colores hexadecimales fijos (`#333333`); siempre se deben invocar variables como `var(--color-primary)` o `var(--bg-surface)`.

Capa 2 — Estructura Maestra y Skeletons (`assets/css/layout.css`) [PROTEGIDO]: Contiene el diseño de la barra lateral (`sidebar`), barra superior (`topbar`), contenedores principales (`grid`/`flexbox`), tarjetas base, tablas institucionales y la animación de carga `.skeleton` (efecto *shimmer* que muestra bloques grises animados mientras `fetch()` obtiene datos del servidor).

Capa 3 — Estilos Específicos del Módulo (`assets/css/modulo.css`) [LIBRE PARA EL EQUIPO]: Es el único archivo CSS donde cada equipo agrega clases propias para las vistas particulares de su módulo, prefijando sus clases con el nombre de su módulo

(por ejemplo, `.oficialia-form-card`, `.audiencias-calendar-grid`, `.peritos-dictamen-badge`) para evitar colisiones.

---

3. ESPECIFICACIÓN DE TECNOLOGÍAS, LIBRERÍAS Y DEPENDENCIAS (`pom.xml`)

3.1. Entorno de Ejecución y Empaquetado

- **Lenguaje y Versión Base:** Java 21 LTS (`maven.compiler.source = 21`, `maven.compiler.target = 21`).
- **Codificación de Caracteres:** `UTF-8` (`project.build.sourceEncoding = UTF-8`) para soporte íntegro de acentos y caracteres propios del español jurídico (`ñ`, `á`, `é`, `í`, `ó`, `ú`).
- **Servidor de Aplicaciones (Servlet Container):** Apache Tomcat `10.1.x` (implementación nativa de **Jakarta EE 10**, utilizando el espacio de nombres `jakarta.servlet.*` en lugar del antiguo `javax.servlet.*`).
- **Empaquetado:** `war` gestionado mediante `org.apache.maven.plugins:maven-war-plugin` versión `3.4.0`.

3.2. Matriz de Dependencias Oficiales y Versiones Exactas

La siguiente tabla detalla todas las librerías declaradas en el archivo `/pom.xml` del núcleo, su versión exacta, su alcance (`scope`) y su propósito técnico dentro del sistema:

GroupId	ArtifactId	Versión	Scope	Propósito Técnico en el Proyecto
`jakarta.servlet`	`jakarta.servlet-api`	`6.0.0`	`provided`	API oficial de Jakarta EE 10 para crear Servlets (`HttpServlet`), Filtros (`Filter`), Listeners (`ServletContextListener`) y manejar sesiones (`HttpSession`). Es `provided` porque Tomcat 10.1 ya la incluye en tiempo de ejecución.
`com.mysql`	`mysql-connector-j`	`8.3.0`	`compile`	Driver JDBC Tipo 4 oficial de MySQL para abrir conexiones hacia `sij_db` desde `ConexionMySQL` y ejecutar transacciones SQL.
`com.google.protobuf`	`protobuf-java`	`3.25.5`	`compile`	Dependencia explícita de seguridad para sobreescribir la versión transitiva de `mysql-connector-j` y mitigar vulnerabilidades CVE conocidas en Protocol Buffers.
`com.google.code.gson`	`gson`	`2.10.1`	`compile`	Librería de Google para serializar objetos Java (POJOs/Listas) a formato JSON y deserializar

GroupId	ArtifactId	Versión	Scope	Propósito Técnico en el Proyecto
				peticiones JSON entrantes en los Servlets API y `JsonResponseUtil`.
`jakarta.mail`	`jakarta.mail-api`	`2.1.2`	`compile`	API estándar de correo electrónico para el envío de notificaciones institucionales y códigos/tokens de recuperación de contraseña desde `EmailService`.
`org.mindrot`	`jbcrypt`	`0.4`	`compile`	Implementación del algoritmo criptográfico de hashing **BCrypt** con *salt* automático para cifrar y verificar contraseñas en `DatabaseInitializer`, `LoginServlet` y `UserAdminServlet`.

---

4. CATÁLOGO DETALLADO DE DIRECTORIOS Y ARCHIVOS DEL PROYECTO (`sij-core-template`)

4.1. Árbol Completo del Proyecto en el IDE

A continuación se presenta la estructura exacta del proyecto base `sij-core-template`:

```
None
sij-core-template/
├── .github/
│   ├── workflows/
│   │   └── build.yml
│   └── CODEOWNERS
├── .mvn/
├── src/main/
│   ├── java/mx/edu/utez/sij/
│   │   ├── controller/
│   │   │   ├── admin/
│   │   │   │   └── UserAdminServlet.java
│   │   │   ├── api/
│   │   │   └── auth/
│   │   ├── core/
│   │   │   ├── auth/
│   │   │   │   ├── LoginServlet.java
│   │   │   │   ├── LogoutServlet.java
│   │   │   │   └── PasswordResetServlet.java
│   │   │   ├── config/
│   │   │   │   ├── ConexionMySQL.java
│   │   │   │   └── DatabaseInitializer.java
│   │   │   ├── filter/
```

```
|   |   |   |   ├── AuthFilter.java
|   |   |   |   └── CorsFilter.java
|   |   |   └── util/
|   |   |       ├── EmailService.java
|   |   |       ├── EmployeeCodeUtil.java
|   |   |       └── JsonResponseUtil.java
|   |   ├── dao/
|   |   ├── model/
|   |   └── service/
|   ├── resources/
|   |   ├── sql/
|   |   |   └── schema.sql
|   |   └── db.properties.example
|   └── webapp/
|       ├── assets/
|       |   ├── css/
|       |   |   ├── layout.css
|       |   |   ├── modulo.css
|       |   |   └── variables.css
|       |   ├── img/
|       |   └── js/
|       ├── views/
|       |   ├── admin/
|       |   |   └── index.html
|       |   └── dashboard/
|       |       └── index.html
|       ├── WEB-INF/
|       |   └── web.xml
|       ├── index.html
|       └── index.jsp
├── .gitignore
├── mvnw
├── mvnw.cmd
├── pom.xml
└── README.md
```

4.2. Raíz del Proyecto y Automatización (`.github/`, `.mvn/`)

- **`.github/workflows/build.yml`:** Pipeline de Integración Continua (GitHub Actions). Cada vez que un integrante abre un Pull Request o hace `push` hacia `develop` o `main`, GitHub levanta una máquina virtual Ubuntu con Java 21 y ejecuta `./mvnw clean compile test package` para verificar automáticamente que el código compila sin errores antes de permitir el merge.

- **`.github/CODEOWNERS`:** Archivo de gobernanza de GitHub que bloquea modificaciones no autorizadas sobre los archivos del núcleo (`core`), exigiendo revisión explícita cuando un PR intenta tocar rutas críticas.
- **`.mvn/`, `mvnw` y `mvnw.cmd`:** *Maven Wrapper*. Permite que cualquier integrante compile el proyecto desde consola o desde IntelliJ IDEA usando exactamente la misma versión de Maven sin necesidad de instalar Maven manualmente en Windows (`mvnw.cmd`) o Linux/macOS (`./mvnw`).
- **`.gitignore`:** Reglas de exclusión de Git. Impide que se suban al repositorio la carpeta de compilación `target/`, configuraciones locales del IDE (`.idea/`, `*.iml`) y **especialmente el archivo `src/main/resources/db.properties`** (para proteger contraseñas locales y credenciales del usuario `ROOT`).

4.3. Paquete Núcleo Inmutable (`src/main/java/mx/edu/utez/sij/core/`)

IMPORTANTE: Ningún equipo debe modificar las clases dentro de `mx.edu.utez.sij.core.*`. Todas están listas para ser utilizadas directamente desde sus módulos.

- **`core/auth/LoginServlet.java` (`/api/auth/login` y `/api/auth/session`):**
    - En `POST /api/auth/login`: Recibe las credenciales (código de empleado o correo institucional y contraseña), consulta el usuario activo en `sij_db`, verifica el hash con `BCrypt.checkpw()`, crea la sesión HTTP (`HttpSession`) almacenando `userId`, `codigoEmpleado`, `nombreCompleto`, `rol` y `moduloAsignado`, y responde en JSON con la ruta de redirección correspondiente.
    - En `GET /api/auth/session`: Devuelve en formato JSON los datos del usuario autenticado actualmente en sesión para que el frontend pinte su nombre, código y permisos en el Dashboard sin recargar la página.
- **`core/auth/LogoutServlet.java` (`/api/auth/logout`):** Invalida de forma segura la sesión actual (`session.invalidate()`), limpia el contexto del usuario y redirige al inicio de sesión (`/index.html`).
- **`core/auth/PasswordResetServlet.java` (`/api/auth/password-reset`):** Gestiona el flujo universal de recuperación de contraseña para personal judicial mediante generación de código/token temporal y envío por correo institucional a través de `EmailService`.
- **`core/config/ConexionMySQL.java`:** Clase centralizada proveedora de conexiones JDBC. Lee las credenciales desde `db.properties` (o variables de entorno en caso de despliegue) y expone el método estático `ConexionMySQL.getConnection()`. Todos los DAOs de todos los módulos **deben** obtener su conexión exclusivamente llamando a `ConexionMySQL.getConnection()`.
- **`core/config/DatabaseInitializer.java` (`@WebListener`):** Clase que implementa `ServletContextListener`. Se ejecuta **automáticamente en cuanto arranca Apache Tomcat**:
    a. Se conecta a MySQL y ejecuta el script `src/main/resources/sql/schema.sql` para crear la base de datos `sij_db` y todas sus tablas si aún no existen.

    b. Verifica si existe un usuario con rol `ROOT` en la tabla `usuarios`. Si la tabla está vacía, lee las propiedades `app.bootstrap.admin.*` de `db.properties`, cifra la contraseña en tiempo real con `BCrypt.hashpw(..., BCrypt.gensalt(12))` e inserta automáticamente el usuario administrador inicial sin exponer hashes ni contraseñas en el código fuente de GitHub.

- **`core/filter/AuthFilter.java` (`@WebFilter("/*")`):** Guardián de seguridad del sistema. Intercepta todas las peticiones entrantes:
    - Permite el paso libre únicamente a rutas públicas (`/index.html`, `/api/auth/*`, `/assets/*`).
    - Verifica que exista una `HttpSession` válida para cualquier vista dentro de `/views/*` o cualquier endpoint privado `/api/*`.
    - Bloquea el acceso a `/views/admin/*` y `/api/admin/*` si el usuario en sesión no posee el rol `ROOT`, devolviendo `403 Forbidden`.
    - Añade cabeceras `Cache-Control: no-cache, no-store, must-revalidate` para impedir que al presionar el botón "Atrás" del navegador después de cerrar sesión se visualicen pantallas previas.
- **`core/filter/CorsFilter.java` (`@WebFilter("/api/*")`):** Configura las cabeceras HTTP de *Cross-Origin Resource Sharing* (`Access-Control-Allow-Origin`, `Access-Control-Allow-Methods`, `Access-Control-Allow-Headers`) en todos los endpoints bajo `/api/*`. Gracias a este filtro, cuando el módulo de Oficialía necesite consultar por HTTP un endpoint del módulo de Audiencias o Peritos que corre en otro puerto o servidor, el navegador permitirá la comunicación sin bloquearla por políticas CORS.
- **`core/util/EmailService.java`:** Servicio utilitario transversal para el envío de correos electrónicos institucionales (alta de cuentas de personal judicial y restablecimiento de contraseñas) leyendo la configuración SMTP desde `db.properties`.
- **`core/util/EmployeeCodeUtil.java`:** Generador estandarizado de identificadores gubernamentales alfanuméricos para servidores públicos del juzgado. Construye el código único de empleado combinando el prefijo del rol/área, las iniciales del nombre completo, el año de alta y un sufijo aleatorio verificable (por ejemplo: `JUEZ-EMG-2025-849`).
- **`core/util/JsonResponseUtil.java`:** Utilidad estándar para responder peticiones desde cualquier Servlet. Expone métodos estáticos como `sendSuccess(response, data, message)` y `sendError(response, statusCode, errorMessage)` que configuran automáticamente `response.setContentType("application/json;charset=UTF-8")` y serializan la estructura estándar `{ "success": true|false, "message": "...", "data": ... }` con `Gson`.

4.4. Paquetes de Trabajo por Módulo (`controller`, `service`, `dao`, `model`)

Aquí es donde cada uno de los 4 equipos desarrollará su lógica de negocio:

- **`controller/admin/UserAdminServlet.java` (`/api/admin/users`) [PROTEGIDO]:** Controlador exclusivo para el usuario `ROOT`. Dado que en un sistema judicial gubernamental **no existe el registro público abierto**, únicamente el usuario `ROOT` puede dar de alta cuentas nuevas para jueces, secretarios de acuerdos, oficiales de partes o peritos desde este controlador, el cual genera su matrícula con `EmployeeCodeUtil` y cifra su contraseña con `BCrypt`.
- **`controller/api/` [ESPACIO DE TRABAJO DEL EQUIPO]:** Carpeta donde cada equipo debe crear sus Servlets de negocio y sus endpoints de comunicación entre módulos. Todos los Servlets que se coloquen aquí deben anotarse con `@WebServlet("/api/<modulo>/<recurso>")` (ejemplo: `@WebServlet("/api/oficialia/demandas")` o `@WebServlet("/api/audiencias/salas")`).
- **`controller/auth/`:** Reservado para extensiones específicas de autorización o validaciones adicionales de sesión si un módulo lo requiere.
- **`service/` [ESPACIO DE TRABAJO DEL EQUIPO]:** Aquí cada equipo crea sus clases de reglas de negocio (ejemplo: `DemandaService.java`, `AudienciaService.java`, `TocaService.java`, `PeritoService.java`). Ningún Servlet debe escribir SQL directamente; el Servlet llama al `Service` y el `Service` llama al `DAO`.
- **`dao/` [ESPACIO DE TRABAJO DEL EQUIPO]:** Aquí cada equipo crea sus clases de acceso a datos (ejemplo: `DemandaDAO.java`). Toda consulta SQL debe usar `try (Connection conn = ConexionMySQL.getConnection(); PreparedStatement ps = conn.prepareStatement(sql))` para cerrar recursos automáticamente y prevenir inyección SQL.
- **`model/` [ESPACIO DE TRABAJO DEL EQUIPO]:** Aquí cada equipo coloca sus clases Java (POJOs) con constructores, `getters` y `setters` que representan las tablas de su módulo (ejemplo: `Demanda.java`, `Expediente.java`, `Audiencia.java`, `Perito.java`).

4.5. Recursos y Base de Datos (`src/main/resources/`)

- **`resources/sql/schema.sql` [PROTEGIDO]:** Script DDL maestro idempotente (`CREATE TABLE IF NOT EXISTS ...` e `INSERT IGNORE ...`). Define el esquema unificado `sij_db` con los catálogos base (`materias`, `juzgados`, `usuarios`, `expedientes`, etc.). Ningún equipo debe modificar este archivo por su cuenta; si un equipo necesita agregar tablas o columnas para su módulo, debe solicitarlo al encargado de soporte de la configuración base (`@angelromero-dev`) para que se actualice de forma centralizada y todos los equipos tengan exactamente el mismo esquema.
- **`resources/db.properties.example` [PROTEGIDO]:** Plantilla de configuración documentada. Muestra las llaves exactas que requiere el sistema (`db.url`, `db.user`, `db.password`, `app.bootstrap.admin.email`, `app.bootstrap.admin.password`, `app.bootstrap.admin.nombre`, y configuración SMTP) sin contener contraseñas reales de producción.

4.6. Capa de Presentación Web (`src/main/webapp/`)

- **`webapp/index.html` [PROTEGIDO]:** Pantalla institucional de Inicio de Sesión (Login) del Poder Judicial. Se comunica de forma asíncrona con `POST /api/auth/login` y redirige automáticamente al usuario según su rol.
- **`webapp/index.jsp`:** Archivo puente de redirección automática hacia `index.html` cuando se accede a la raíz del contexto.
- **`webapp/WEB-INF/web.xml`:** Descriptor de despliegue estándar de Jakarta EE 10 (configuración de bienvenida, tiempo de expiración de sesión de 30 minutos y mapeo de errores HTTP).
- **`webapp/views/admin/index.html` [PROTEGIDO]:** Panel exclusivo para el rol `ROOT` donde se registran y administran los usuarios del juzgado (altas de personal judicial, asignación de rol y módulo, activación/baja lógica).
- **`webapp/views/dashboard/index.html`:** Vista principal posterior al inicio de sesión (Dashboard Universal y contenedor de trabajo del módulo), equipada con estructura de carga progresiva (`.skeleton`) y consumo de `/api/auth/session`. Cada equipo puede añadir subcarpetas dentro de `webapp/views/<su-modulo>/` para sus pantallas secundarias.
- **`webapp/assets/css/variables.css` y `layout.css` [PROTEGIDOS]:** Sistema de diseño global explicado en la Sección 2.3.
- **`webapp/assets/css/modulo.css` [ESPACIO DE TRABAJO DEL EQUIPO]:** Hoja de estilos propia del módulo.
- **`webapp/assets/js/` [ESPACIO DE TRABAJO DEL EQUIPO]:** Carpeta donde cada equipo coloca sus scripts JavaScript modulares para validaciones de formularios, consumo de endpoints con `fetch()` y renderizado dinámico del DOM.
- **`webapp/assets/img/`:** Logotipos e iconografía institucional del sistema.

---

5. POLÍTICA DE ARCHIVOS PROTEGIDOS Y GOBERNANZA COLABORATIVA (`.github/CODEOWNERS`)

5.1. Propósito de la Protección del Core

Para evitar que un cambio accidental en la conexión a base de datos, en las hojas de estilo globales o en el filtro de sesión deje sin funcionar a los demás integrantes o genere conflictos masivos al integrar los módulos, el archivo `.github/CODEOWNERS` protege los archivos base del repositorio.
Esto es para asegurar que el núcleo (`core`) se mantenga estable y homogéneo entre los 4 repositorios. Por su parte, **el representante de cada equipo** es quien revisa y autoriza los Pull Requests funcionales correspondientes a las tareas de su propio módulo (`controller/api`, `service`, `dao`, `model`, `assets/js`, `assets/css/modulo.css`).

5.2. Tabla Oficial de Rutas Protegidas y Procedimiento de Solicitud de Cambios

Ruta Protegida en `.github/CODEOWNERS`	Motivo de la Protección	¿Cómo solicitar un cambio o adición?
`/pom.xml`	Evitar conflictos de versiones de librerías o dependencias incompatibles con Tomcat 10.1 / Java 21.	El representante del equipo solicita la librería indicando `groupId`, `artifactId`, `version` y justificación técnica.
`/src/main/java/mx/edu/utez/sij/core/`	Proteger el Login, Filtros de Sesión/CORS, `ConexionMySQL`, `DatabaseInitializer` y utilidades transversales.	Si se detecta un bug o se requiere un nuevo método utilitario transversal, abrir un Issue o notificar al encargado de soporte de la configuración base.
`/src/main/resources/sql/schema.sql`	Mantener una única fuente de verdad (*Single Source of Truth*) de la base de datos `sij_db` para los 4 equipos.	El equipo diseña su propuesta de tabla `CREATE TABLE` con llaves foráneas, la valida con su representante de equipo y la entrega para integrarla en el `schema.sql` oficial.
`/src/main/webapp/assets/css/variables.css`	Preservar la paleta institucional del Poder Judicial y los tokens de diseño globales.	Usar las variables existentes en `modulo.css`. Si falta un token global, solicitar su inclusión.
`/src/main/webapp/assets/css/layout.css`	Evitar que se deforme el Sidebar, Topbar o los componentes base compartidos.	Agregar cualquier estilo particular del módulo dentro de `assets/css/modulo.css`.
`/src/main/webapp/index.html`	Garantizar que el inicio de sesión único funcione igual para todos los roles del juzgado.	No requiere modificación por parte de los módulos.
`/src/main/resources/db.properties.example`	Mantener estandarizadas las llaves de configuración local para todos los desarrolladores.	Si se añade una nueva variable de entorno global, se actualiza de forma centralizada.
`/src/main/java/mx/edu/utez/sij/controller/admin/UserAdminServlet.java`	Proteger el alta gubernamental de cuentas y generación de códigos de empleado.	Si un módulo requiere un nuevo rol en el catálogo de alta, solicitar su incorporación.
`/src/main/webapp/views/admin/index.html`	Resguardar la interfaz de administración de usuarios del rol `ROOT`.	No requiere modificación por parte de los módulos.

---

6. CONFIGURACIÓN DEL ENTORNO LOCAL, BASE DE DATOS Y PRUEBAS DE INTEGRACIÓN

6.1. Configuración de `db.properties` y Semilla Automática (*Bootstrap Admin*)

Por seguridad industrial (estándar *Twelve-Factor App*), **jamás se suben contraseñas ni usuarios reales al repositorio de GitHub**. Al clonar el repositorio por primera vez, cada desarrollador debe seguir estos 3 pasos en su máquina:

11. Ir a la carpeta `src/main/resources/`.
12. Duplicar el archivo `db.properties.example` y renombrar la copia exactamente como **`db.properties`** (este archivo ya está ignorado en `.gitignore`).
13. Abrir `src/main/resources/db.properties` y colocar sus credenciales locales de MySQL y los datos de prueba para su usuario `ROOT` local:

```
None
# 1. Conexión local a MySQL (El esquema sij_db se crea automáticamente si no existe)
db.url=jdbc:mysql://localhost:3306/sij_db?createDatabaseIfNotExist=true&useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=America/Mexico_City
db.user=root
db.password=tu_password_local_de_mysql

# 2. Usuario Administrador Inicial (Bootstrap ROOT - Solo se inserta si la tabla usuarios está vacía)
app.bootstrap.admin.codigo=ROOT-ADMIN-0001
app.bootstrap.admin.nombre=Administrador General del Sistema Judicial
app.bootstrap.admin.email=admin@sij.morelos.gob.mx
app.bootstrap.admin.password=AdminSij2025!
app.bootstrap.admin.rol=ROOT
```

¿Cómo funciona esto al entregar al cliente?
Cuando levantas Apache Tomcat por primera vez, la clase `DatabaseInitializer` lee tu `db.properties` local, ejecuta `schema.sql` para crear todas las tablas y, al detectar que no existe ningún usuario `ROOT`, toma `app.bootstrap.admin.password`, la encripta con **BCrypt** en ese instante y crea el usuario administrador en tu MySQL local. Cuando el sistema se entrega en el servidor del cliente (o del juzgado), el administrador de sistemas (SysAdmin/DevOps) coloca su propio `db.properties` o variables de entorno en el servidor con una contraseña confidencial que ni siquiera los programadores conocen.

6.2. Cómo Levantar y Probar el Proyecto en Apache Tomcat 10.1.x

14. Abrir el proyecto en **IntelliJ IDEA Ultimate** (o Community con plugin Smart Tomcat) verificando que el SDK del proyecto sea **JDK 21**.
15. Configurar una ejecución de **Apache Tomcat 10.1.x**:
    - En la pestaña *Deployment*, agregar el artefacto `sij-core:war exploded`.

    - Definir el *Application context* como `/` (o `/sij-oficialia`, `/sij-audiencias`, etc., según se desee probar).
16. Iniciar el servidor Tomcat. En la consola de IntelliJ verás el mensaje de `[SIJ-CORE] Esquema sij_db inicializado correctamente` y `[SIJ-CORE] Usuario ROOT inicial verificado`.
17. Abrir el navegador en `http://localhost:8080/` e iniciar sesión con el correo y contraseña definidos en tu `db.properties`.
18. Al entrar como `ROOT`, ir al panel de administración de usuarios (`/views/admin/index.html`) para crear usuarios de prueba de tu módulo (por ejemplo, rol `OFICIAL_PARTES`, `SECRETARIO_AUDIENCIAS`, `MAGISTRADO_TOCAS` o `PERITO`).

6.3. Cómo Probar Endpoints entre Módulos Distintos (Postman, Multi-WAR y `ngrok`)

Existen **3 mecanismos estándar** que se utilizan según la etapa del desarrollo:

19. **Etapa 1 — Desarrollo Individual con Contratos JSON (Mocking / Postman):**
    Como todos los módulos comparten la misma base de datos local `sij_db` en tu computadora (gracias a `schema.sql`), tu módulo puede consultar directamente las tablas maestras compartidas (`expedientes`, `materias`, `juzgados`). Para probar tus propios endpoints (`GET /api/oficialia/expedientes`, `POST /api/oficialia/demandas`), utiliza **Postman** o **cualquier cliente** enviando peticiones JSON a `http://localhost:8080/api/...` y verificando que `JsonResponseUtil` responda con código HTTP `200` o `201`.
20. **Etapa 2 — Prueba Local Multi-Módulo en un mismo Tomcat (Multi-WAR):**
    Tomcat permite desplegar varios archivos `.war` al mismo tiempo en la misma computadora bajo distintos contextos:
    - `http://localhost:8080/oficialia/api/...`
    - `http://localhost:8080/audiencias/api/...`
    - `http://localhost:8080/peritos/api/...`
    - `http://localhost:8080/tocas/api/...`

Solo necesitas parametrizar la URL base de los módulos en un archivo de configuración JS o propiedades para apuntar al contexto correspondiente.

21. **Etapa 3 — Pruebas en Vivo entre Computadoras de Distintos Equipos (`ngrok` o Red Local LAN):**
    Cuando un integrante del equipo de Oficialía quiera enviar una petición HTTP en tiempo real hacia el servidor de un compañero del equipo de Audiencias que está en otra ubicación:
    - El compañero de Audiencias levanta su Tomcat en el puerto `8080` y ejecuta en su terminal: `ngrok http 8080`.
    - `ngrok` le generará una URL pública segura temporal (ejemplo: `https://a1b2-c3d4.ngrok-free.app`).

- El compañero de Oficialía coloca esa URL como destino del endpoint (`https://a1b2-c3d4.ngrok-free.app/api/audiencias/programar`) y, gracias a que `CorsFilter.java` ya está configurado en el `core`, ambos módulos se comunicarán por internet en tiempo real sin necesidad de pagar servidores en la nube.

---

7. ESTRATEGIA DE RAMAS EN GIT (GITFLOW ADAPTADO) Y DIVISIÓN DEL TRABAJO SIN CONFLICTOS

7.1. Propósito de las Ramas Principales (`main` y `develop`)

En ningún repositorio se permite hacer `git push` directo a `main` ni a `develop`. Ambas ramas están protegidas por reglas de repositorio en GitHub:

- **`main` (Rama de Producción / Entregas Oficiales):** Contiene exclusivamente versiones 100% terminadas, probadas y estables listas para ser presentadas en revisión académica o despliegue final. Solo recibe código proveniente de `develop` al cierre de un corte o entrega.
- **`develop` (Rama de Integración Continua del Equipo):** Es la rama base de trabajo diario del módulo. Aquí se van uniendo (mediante Pull Requests) todas las funcionalidades terminadas por los miembros del equipo a lo largo de la semana. Cuando un integrante va a iniciar una tarea nueva, **siempre crea su rama a partir de `develop` actualizada**.

7.2. Nomenclatura Estándar de Ramas por Equipo, Integrante y Tarea

Para identificar de inmediato qué módulo, qué integrante y qué funcionalidad contiene cada rama, todas las ramas de trabajo deben seguir estrictamente la siguiente estructura en minúsculas y separada por guiones medios (`kebab-case`):

```
None
[<iniciales>]<tipo>/<ámbito>: <descripcion corta tarea>
```

- **`<tipo>`:** Propósito de la rama (`feat` para nueva funcionalidad, `fix` para corrección de errores, `refactor` para mejora de código, `docs` para documentación).
- **`<ámbito>`:** Acota la parte del código afectada (ej. `auth`, `ui`, `api`).
- **`<iniciales>`:** Iniciales del nombre completo del integrante responsable de esa sub-rama entre corchetes [] (ejemplo: `ajrp`, `mlg`, `jch`).
- **`<descripcion-corta-tarea>`:** De 2 a 4 palabras separadas por guiones que describan la tarea específica del tablero Kanban.

Ejemplos reales de nombres de ramas correctos:

- `[mlg]feat/auth: Implementación de...`

7.3. Cómo Dividir las Tareas en el Equipo para Evitar Choques (Vertical Slicing)

El error más común en equipos estudiantiles es dividir el trabajo "por capas horizontales" (por ejemplo: *"Persona 1 hace todos los DAOs, Persona 2 hace todos los Servlets y Persona 3 hace todos los HTML"*). Eso provoca retrasos (nadie puede probar hasta que el otro termine) y choques constantes en los mismos archivos.

Regla de División Recomendada (Corte Vertical por Caso de Uso / Entidad):

El representante de cada equipo debe asignar a cada integrante un **caso de uso o sub-proceso completo de principio a fin** o bien archivos separados con nombres distintos:

- **Integrante A (Rama `[ajrp]feat/registro-demanda`):** Trabaja en `Demanda.java`, `DemandaDAO.java`, `DemandaService.java`, `DemandaServlet.java` y `assets/js/demandas.js`.
- **Integrante B (Rama `[jch]feat/catalogo-anexos`):** Trabaja en `Anexo.java`, `AnexoDAO.java`, `AnexoService.java`, `AnexoServlet.java` y `assets/js/anexos.js`.
- **Integrante C (Rama `[mlg]feat/bandeja-turnos`):** Trabaja en `TurnoDAO.java`, `TurnoService.java`, `TurnoServlet.java` y `views/oficialia/turnos.html`.

Como cada integrante crea sus propias clases `.java` y su propio archivo `.js`, al unir sus ramas hacia `develop` **Git fusionará los archivos automáticamente con cero conflictos**.

7.4. Reglas de Pull Requests (PR) y Autorización por el Representante de Equipo

22. Ningún integrante puede hacer merge de su propia rama hacia `develop` sin revisión previa.
23. Antes de abrir un Pull Request en GitHub, el integrante debe sincronizar su rama local con los últimos cambios de `develop`:

```shell
git checkout develop
git pull origin develop
git checkout [ajrp]feat/registro-demanda
git merge develop
```

24. Una vez verificado que el proyecto compila localmente y funciona en Tomcat, sube su rama (`git push origin feat/oficialia-ajrp/registro-demanda`) y abre el **Pull Request** en GitHub apuntando **Base: `develop`** ← **Compare: `[ajrp]feat/registro-demanda`**.
25. GitHub Actions ejecutará automáticamente `.github/workflows/build.yml`. Si el check en verde pasa:
    - **El representante del equipo** revisa el código de sus compañeros de módulo y es el único facultado dentro de su equipo para **autorizar (Approve) y fusionar (Merge)** el Pull Request hacia `develop`.
    - Si el Pull Request incluye alguna modificación previamente acordada sobre archivos del núcleo o de la base de datos (`schema.sql`, `pom.xml`, `core/*`), GitHub solicitará automáticamente la confirmación adicional mediante `.github/CODEOWNERS`.

---

8. ESTÁNDAR DE COMMITS: GUÍA COMPLETA DE CONVENTIONAL COMMITS

8.1. Estructura Formal y Uso de Corchetes `[scope]`

Quedan estrictamente prohibidos los mensajes de commit ambiguos como *"cambios"*, *"ya quedó"*, *"subiendo avances"*, *"fix"* o *"actualización"*. Todo el proyecto utiliza el estándar internacional **Conventional Commits** reforzado con un alcance entre corchetes (`[scope]`) para tener trazabilidad exacta en el historial de Git.

Sintaxis obligatoria de cada commit:

```
None
[<iniciales>]<tipo>/<ámbito>: <descripcion corta tarea>
```

8.2. Catálogo de Etiquetas (`feat`, `fix`, `style`, `refactor`, `docs`, `chore`, `test`)

Etiqueta	Significado	¿Cuándo se debe utilizar?
`feat`	*Feature* (Nueva funcionalidad)	Cuando agregas código nuevo que aporta una función al usuario o al sistema (un nuevo Servlet, un nuevo método CRUD en el DAO, una nueva vista HTML o integración con `fetch`).
`fix`	*Bug Fix* (Corrección de error)	Cuando corriges un fallo, excepción (`NullPointerException`, `SQLException`), cálculo erróneo o validación rota en código que ya existía.
`style`	*Styling / Formatting* (Diseño y formato)	Cuando realizas cambios puramente visuales en `modulo.css`, ajustes de maquetación HTML, indentación o formato sin alterar ni una sola regla de negocio ni SQL.
`refactor`	*Code Refactoring* (Reestructuración)	Cuando mejoras la limpieza, legibilidad o estructura interna de un método Java o JS sin cambiar su comportamiento externo ni corregir un bug.
`docs`	*Documentation* (Documentación)	Cuando creas o actualizas archivos `README.md`, comentarios Javadoc o manuales técnicos del repositorio.
`chore`	*Chores / Maintenance* (Mantenimiento)	Cuando actualizas archivos de configuración, `.gitignore`, dependencias en `pom.xml`, scripts de construcción o estructura de paquetes vacíos.
`test`	*Testing* (Pruebas)	Cuando agregas o modificas pruebas unitarias o scripts de prueba de endpoints.

---

9. METODOLOGÍA ÁGIL SCRUMBAN / KANBAN PARA LA GESTIÓN DE EQUIPOS

9.1. Estructura del Tablero Kanban de 5 Columnas y Límites WIP

Para organizar el trabajo semanal sin sobrecargar a los equipos con ceremonias burocráticas pero manteniendo el orden visual de Scrum y el flujo continuo de Kanban, utilizaremos **Scrumban** apoyado en **GitHub Projects** (o Trello/Jira). Cada repositorio o equipo gestiona su tablero con **5 columnas obligatorias**:

26. **Backlog (Pila del Producto):** Todas las historias de usuario y tareas técnicas pendientes del módulo para el cuatrimestre.
27. **Ready / Por Hacer (Compromiso de la Semana):** Tareas priorizadas y completamente claras que el equipo se compromete a desarrollar durante la semana actual. Cada tarjeta debe tener asignado un responsable y el nombre de la rama que usará.
28. **In Progress / En Progreso (Límite WIP = Máximo 1 o 2 tareas por integrante):** Tareas que se están programando activamente en este momento. **Regla de Oro del WIP (Work In Progress):** Ningún integrante puede tener 4 tareas a medias en esta columna; primero termina una, la pasa a revisión y luego toma la siguiente.
29. **In Review / En Revisión (Pull Request Abierto):** Tareas cuyo código ya fue subido a GitHub, tienen un Pull Request abierto hacia `develop`, pasaron la compilación automática de GitHub Actions y están esperando la revisión del representante del equipo.
30. **Done / Terminado:** Tareas cuyo Pull Request ya fue aprobado y fusionado (*merged*) dentro de la rama `develop` y verificadas en ejecución.

9.2. Flujo de una Tarea desde el Backlog hasta el Merge en `develop`

31. El integrante toma su tarjeta asignada de la columna **Ready** y la mueve a **In Progress**.
32. Crea su rama local desde `develop` siguiendo la nomenclatura de la Sección 7.2.
33. Programa su funcionalidad respetando las capas (`Model` → `DAO` → `Service` → `Servlet API` → `Vista/JS`) haciendo commits pequeños y atómicos con **Conventional Commits**.
34. Prueba localmente en Tomcat 10.1 que no existan errores en consola, sube su rama a GitHub, abre su **Pull Request** y mueve su tarjeta a la columna **In Review / PR**.
35. El representante de su equipo revisa los archivos cambiados en GitHub, aprueba el PR, realiza el *Merge* hacia `develop` y mueve la tarjeta a **Done**.

---

10. GUÍA DE BUENAS PRÁCTICAS Y REGLAS DE ORO DE CODIFICACIÓN

36. **Cero Credenciales en el Código Fuente:** Nunca escribas contraseñas de MySQL ni de correo electrónico dentro de clases `.java` ni subas el archivo `db.properties`. Todo

parámetro sensible se lee desde `src/main/resources/db.properties` (ignorado por Git).

37. **Prohibido Concatenar Cadenas en SQL (Prevención de SQL Injection):**
    - **PROHIBIDO:** `String sql = "SELECT * FROM expedientes WHERE folio = '" + folio + "'";`
    - **OBLIGATORIO:** Usar siempre signos de interrogación `?` con `PreparedStatement` y asignar valores con `ps.setString(1, folio);`, `ps.setInt(2, id);`.
38. **Cierre Automático de Conexiones (`try-with-resources`):** En cada método del paquete `dao`, envuelve siempre la obtención de `Connection`, `PreparedStatement` y `ResultSet` dentro de un bloque `try (...) { ... }`. Si dejas conexiones abiertas sin cerrar, el servidor MySQL colapsará por límite de conexiones tras unas cuantas peticiones.
39. **Respuestas JSON Homogéneas:** Todos los Servlets dentro de `controller/api/` deben devolver sus resultados invocando `JsonResponseUtil.sendSuccess(...)` o `JsonResponseUtil.sendError(...)`. Nunca armes cadenas JSON a mano concatenando `String`.